CSRF Where Token Validation Depends on Request Method

PortSwigger Web Security Academy Lab Report

Web Application Penetration Testing

Platform	PortSwigger Web Security Academy
Category	Cross-Site Request Forgery (CSRF)
Tooling	Burp Suite Community Edition, web browser
Status	Completed

1 Objective

The objective was to test whether the application's CSRF protection was consistently enforced across different HTTP request methods. The lab focused on an email-change function and whether changing the request method could avoid the token check.

2 Background

A CSRF attack attempts to make a user's browser send an unwanted request to an application where that user is already signed in. A CSRF token is commonly used to help the application distinguish legitimate requests from forged ones.

For this lab, the important observation was that the application treated POST and GET requests differently when checking the CSRF token.

3 Methodology

1. Signed in to the lab application and inspected the account email-change functionality.
2. Intercepted the email-change request using Burp Suite.
3. Sent the request using the POST method without a CSRF token. The application rejected it and returned the message `Missing parameter 'csrf'`.
4. Changed the request method to GET and observed that the application then reported `Missing parameter 'email'`.
5. This response indicated that the GET route processed the email parameter without requiring the CSRF token that had been required for POST.
6. Prepared an HTML form that submitted the email-change request using GET, with the desired email address supplied as a hidden field.
7. Used the lab's exploit server to store and deliver the proof-of-concept page to the lab victim.
8. Confirmed that the lab displayed the successful completion message.

4 Technical Evidence

4.1 POST request rejected without a token

The intercepted POST request was rejected because the CSRF parameter was missing. This showed that token validation was applied to this request method.



Figure 1: POST request rejected because the CSRF parameter was missing.

4.2 GET request behaviour

After changing the method to GET, the response complained about the missing email parameter rather than a missing CSRF token. This was evidence that the token requirement differed by request method.



Figure 2: GET request response identifying the missing email parameter.

4.3 Proof-of-concept HTML

The exploit page used a form targeting the account email-change route. The email address was included in a hidden input, and JavaScript submitted the form automatically.

```
<form action="https://LAB-ID.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="attacker-controlled@example.net">
```

```
</form>
<script>
  document.forms[0].submit();
</script>
```

The lab-specific host and test email shown in the original evidence were used in the actual exercise. The example above uses placeholders.

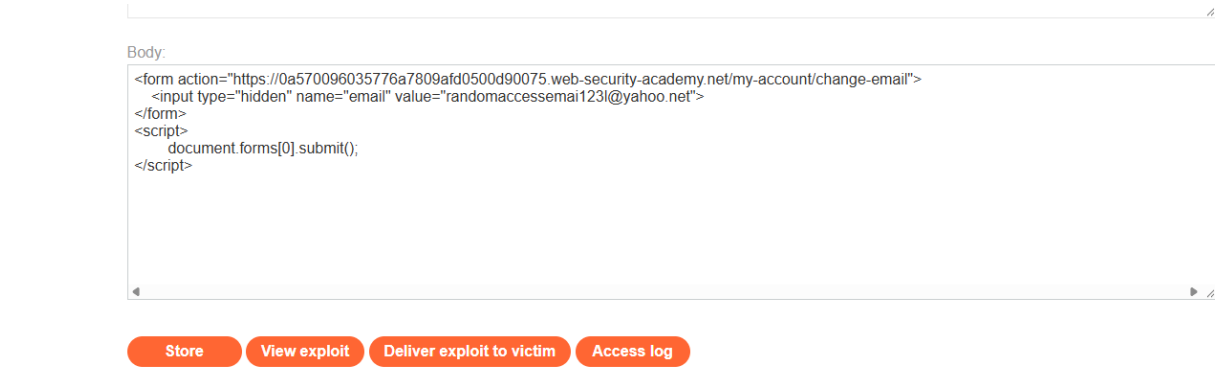


Figure 3: HTML form prepared on the exploit server.

5 Result

The lab was successfully completed. The evidence demonstrates that the application required a CSRF token for the POST request but accepted the email-change operation through the GET route without the same token check. The exploit page used this difference to submit the state-changing request.

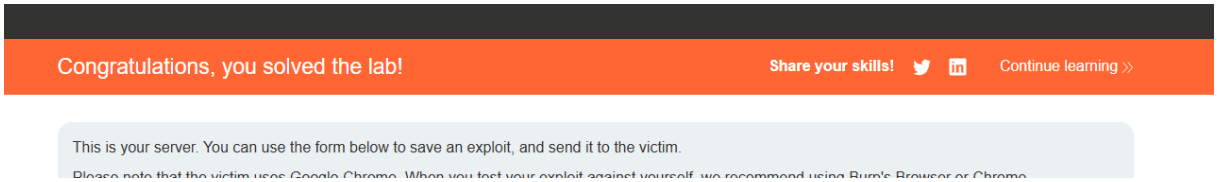


Figure 4: PortSwigger confirmation that the lab was solved.

6 Security Impact

If a real application allows a state-changing action through a request method that does not receive the same CSRF protection, an attacker may be able to cause a signed-in user's browser to perform that action without the user's intention.

In this example, the affected action was changing the account email address. Depending on the wider application, changing an email address could affect account notifications or account recovery.

7 Recommendations

- Apply CSRF protection consistently to every state-changing operation, regardless of the HTTP method.

- Do not use GET requests for actions that change account data or application state.
- Validate CSRF tokens on the server for all relevant state-changing requests.
- Use appropriate cookie settings, including `SameSite`, as an additional layer of protection rather than as the only defence.
- Ensure that changing an account email address requires appropriate confirmation where suitable.

8 Conclusion

This lab demonstrated that CSRF protection can be bypassed when token validation depends on the HTTP request method. The POST request was checked for a CSRF token, while the GET route allowed the same account-change operation without equivalent protection. Security checks should be consistent across all routes and methods that can change application data.